Recuperatorio: Conversión de Catálogo de Videojuegos a React

Introducción

El objetivo de este proyecto es tomar una aplicación existente de visualización de videojuegos, llamada **GameHub**, desarrollada en **HTML, CSS y JavaScript**, y convertirla a **React** utilizando **Vite**. Deberán seguir las mejores prácticas y patrones de diseño de React, aprovechando los hooks y herramientas clave, tales como `useState`, `useEffect`, `useParams`, `react-router`, `useNavigate`, `useContext`, y **Custom Hooks**.

Archivos de Datos

Los archivos JSON con la información de videojuegos y usuarios se encuentran en la carpeta `data/`:

- `data/videojuegos.json` - Contiene el catálogo completo de videojuegos **agrupados por plataforma** (PC, PlayStation, Xbox, Nintendo). Cada plataforma contiene un array de videojuegos con sus DLCs incluidos y géneros. **IMPORTANTE**: La estructura es un objeto con claves de plataforma, no un array plano.
- `data/usuarios.json` - Contiene los usuarios registrados para autenticación

IMPORTANTE: Al convertir a React con Vite, estos archivos deben moverse a la carpeta `public/data/` para que sean accesibles mediante fetch.

Estructura del JSON de videojuegos:

```
{
  "PC": [
    { videojuego1 },
    { videojuego2 },
    ...
  ],
  "PlayStation": [...],
  "Xbox": [...],
  "Nintendo": [...]
}
```

Nota: Los videojuegos NO tienen la propiedad `plataforma` ya que está implícita en la estructura del objeto.

Requerimientos Técnicos

La conversión debe implementar lo siguiente:

- **useState**: Para manejar el estado de los videojuegos, usuarios, favoritos, búsqueda y otros datos relevantes.
- **useEffect**: Para manejar la carga inicial de datos (como los videojuegos desde el archivo JSON) y la actualización del componente.
- **useParams**: Para leer parámetros de la URL en las rutas dinámicas (`/plataforma/:nombre` y `/videojuego/:id`).
- **react-router**: Para manejar la navegación entre las diferentes páginas.
- **useNavigate**: Para redirigir a los usuarios entre las diferentes páginas.
- **useContext**: Para manejar el estado global de autenticación del usuario Y el sistema de favoritos. El contexto debe incluir tanto la información del usuario autenticado como la lista de videojuegos favoritos.
- **Custom Hooks**:
  - **useAuth**: Para encapsular la lógica de autenticación de usuarios.
  - **useFavoritos**: Para manejar la lógica de agregar/quitar favoritos (debe usar el contexto de favoritos).

Pasos para la Conversión

1. Inicializar el Proyecto con Vite

- Crea un nuevo proyecto utilizando **Vite**.
- Organiza la estructura del proyecto en carpetas adecuadas (`components`, `hooks`, `context`, `utils`, etc.).
- **Crea la carpeta** `public/data/` y copia los archivos `videojuegos.json` y `usuarios.json` dentro de ella.

Ejemplo de Fetch en React:

En React con Vite, los archivos en la carpeta `public/` son accesibles directamente. Para cargar los datos:

```
// Ejemplo de carga de videojuegos (estructura agrupada por plataforma)
useEffect(() => {
  const fetchVideojuegos = async () => {
    try {
      const response = await fetch('/data/videojuegos.json');
      const data = await response.json();
      // data es un objeto con claves: PC, PlayStation, Xbox, Nintendo
      // Cada clave contiene un array de videojuegos
      setVideojuegos(data);

      // Para obtener todos los videojuegos en un array plano:
      // const todosLosVideojuegos = Object.values(data).flat();

      // Para obtener videojuegos de una plataforma específica:
      // const videojuegosPC = data.PC;
    } catch (error) {
      console.error('Error al cargar videojuegos:', error);
    }
  };

  fetchVideojuegos();
}, []);
```

Nota: La ruta comienza con `/data/` (no `./data/` ni `../data/`) porque Vite sirve automáticamente el contenido de `public/` desde la raíz.

2. Crear las Rutas con `react-router`

- Implementa el enrutamiento con **react-router**:
  - Ruta principal (`/`): Donde se mostrarán los videojuegos disponibles con opciones de filtrado por plataforma y búsqueda.
  - Ruta de plataforma (`/plataforma/:nombre`): Donde se mostrarán solo los videojuegos de una plataforma específica (PC, PlayStation, Xbox, Nintendo). Debe usar `useParams` para obtener el nombre de la plataforma.
  - Ruta de detalles del videojuego (`/videojuego/:id`): Donde se mostrará la información completa del videojuego seleccionado. Debe usar `useParams` para obtener el ID.
  - **Ruta 404 (`*`)**: Para manejar páginas no encontradas. Debe mostrar un mensaje de error temático y un botón para volver al inicio.

3. Página Principal (Componente `Home`)

- **useState**: Para manejar la lista de videojuegos, término de búsqueda, plataforma seleccionada y estado de carga.
- **useEffect**: Para cargar los videojuegos desde el archivo `videojuegos.json` . **IMPORTANTE**: El JSON viene agrupado por plataforma, por lo que deberás procesar la estructura del objeto para trabajar con los datos.
- **useNavigate**: Al hacer clic en un videojuego, navega a la página de detalles usando `useNavigate()` . Al hacer clic en una plataforma, navega a `/plataforma/:nombre` .
- Debe incluir:
  - **Componente de búsqueda**: Campo de texto para buscar videojuegos por nombre o desarrollador (obligatorio).
  - **Filtros por plataforma**: Botones o enlaces para filtrar por PC, PlayStation, Xbox, Nintendo (deben navegar a `/plataforma/:nombre`).
  - Cards de videojuegos organizadas por plataforma.
  - Indicador visual si un videojuego está en favoritos (corazón o estrella).

4. Página de Plataforma (Componente `PlataformaPage`) - NUEVO

- **useParams**: Para obtener el nombre de la plataforma desde la URL (`/plataforma/:nombre`).
- **useState**: Para manejar la lista de videojuegos filtrados y estado de carga.
- **useEffect**: Para cargar los videojuegos y extraer solo los de la plataforma obtenida de `useParams` (ejemplo: `data[plataforma]`).
- Debe mostrar solo los videojuegos de la plataforma seleccionada accediendo directamente a `data[plataforma]` .
- Incluir un botón para **volver al inicio**.
- Validar que la plataforma existe, si no existe mostrar mensaje de error y opción de volver.

5. Página de Detalles del Videojuego (Componente `VideojuegoDetail`)

- **useParams**: Para obtener el ID del videojuego desde la URL (`/videojuego/:id`).
- **useState**: Para manejar los detalles del videojuego y estado de carga.
- **useEffect**: Para cargar todos los videojuegos y buscar el videojuego específico por ID en todas las plataformas (deberás iterar sobre `Object.values(data).flat()` para encontrar el videojuego con el ID correspondiente).
- Muestra la información completa del videojuego:
  - Imagen
  - Nombre y desarrollador
  - Descripción
  - Año de lanzamiento, género y calificación
  - **Información de los DLCs incluidos** (nombre, tipo, contenido adicional, precio, descripción)
  - Requisitos del sistema
  - Modos de juego disponibles
  - Características destacadas
  - **Botón de favorito**: Permitir agregar/quitar el videojuego de favoritos (debe usar el contexto de favoritos).
- Incluye un botón para **volver al catálogo**.

6. Página 404 (Componente `NotFound`)

- Crea un componente para manejar rutas no encontradas.
- Debe mostrar:
  - El logo de **GAMEHUB** (🎮).
  - Un mensaje de error **404 - Página no encontrada**.
  - Una descripción amigable del error relacionada con la temática (ejemplo: "Parece que este juego se perdió en el multiverso 🎮").
  - Un botón para **volver al inicio** usando `useNavigate()` .
- Mantén la estética consistente con el resto de la aplicación (colores gaming: azul oscuro #0A0E27, verde neón #00FF88, blanco).

7. Sistema de Autenticación y Favoritos con Context

- **useContext**: Crea un contexto (`AppContext`) que maneje:
  - **Estado de autenticación del usuario**: nombre, rol, si está autenticado.
  - **Sistema de favoritos**: lista de IDs de videojuegos favoritos.
  - Funciones: `login` , `logout` , `agregarFavorito` , `quitarFavorito` , `esFavorito` .
- El formulario de inicio de sesión debe mostrarse si el usuario no ha iniciado sesión.
- Una vez autenticado, debe actualizarse el componente de login para mostrar el nombre completo, el rol y el botón de cerrar sesión.
- Los favoritos deben persistir durante la navegación entre páginas (usar localStorage es opcional pero recomendado).
- La información debe persistir durante la navegación entre páginas.

8. Implementación de Custom Hooks

- **useAuth**: Un custom hook para manejar la lógica de autenticación de usuarios.
  - Validación contra el archivo `usuarios.json` .
  - Manejo del estado de autenticación.
  - Funciones de login y logout.
  - Debe usar el contexto `AppContext` .
- **useFavoritos**: Un custom hook para manejar la lógica de favoritos.
  - Funciones: `agregarFavorito` , `quitarFavorito` , `esFavorito` .
  - Debe usar el contexto `AppContext` para acceder a los favoritos.
  - Retorna las funciones y el estado de favoritos.

9. Componente de Búsqueda (Obligatorio)

- Crea un componente `Buscador` que permita buscar videojuegos por nombre o desarrollador.
- Debe usar `useState` para manejar el término de búsqueda.
- Debe filtrar los videojuegos en tiempo real mientras el usuario escribe.
- Debe mostrarse en la página principal (`Home`).

Requerimientos Funcionales

1. **Visualización de videojuegos**: La lista de videojuegos debe cargarse dinámicamente desde el archivo `videojuegos.json` ubicado en `public/data/` . **IMPORTANTE**: El JSON viene agrupado por plataforma como un objeto con claves `PC` , `PlayStation` , `Xbox` ,

`Nintendo`. Debes procesar esta estructura para mostrar los videojuegos organizados por plataforma.

2. **Búsqueda de videojuegos**: Debe existir un componente de búsqueda que permita filtrar por nombre o desarrollador en tiempo real.
3. **Navegación por plataforma**: Al hacer clic en una plataforma, debe navegar a `/plataforma/:nombre` y mostrar solo los videojuegos de esa plataforma usando `useParams`.
4. **Autenticación de usuarios**: El sistema debe permitir iniciar sesión usando los datos de `usuarios.json` ubicado en `public/data/`. Debe mostrar el formulario de login o la información del usuario (nombre completo y rol) según corresponda. Al iniciar sesión, solo debe actualizarse el componente de login, sin redirecciones.
5. **Sistema de favoritos**: Los usuarios autenticados pueden agregar/quitar videojuegos de sus favoritos. Los favoritos deben mostrarse visualmente en las cards y en la página de detalles.
6. **Navegación**: Implementar navegación entre la página principal, página de plataforma, detalles de videojuego y página 404.
7. **Página 404**: Debe mostrar un mensaje de error amigable y temático cuando se acceda a una ruta inexistente, con opción de volver al inicio.
8. **Hooks personalizados**: Se deben implementar hooks personalizados para la lógica de autenticación y favoritos.
9. **Responsive**: La aplicación debe verse correctamente en dispositivos móviles y desktop.
10. **Información de DLCs**: Cada videojuego debe mostrar claramente la información de los DLCs incluidos (nombre, tipo, contenido adicional, precio y descripción).

Estructura Sugerida del Proyecto React

```
gamehub-react/
├── public/
│   └── data/
│       ├── videojuegos.json
│       └── usuarios.json
├── src/
│   ├── components/
│   │   ├── Header.jsx
│   │   ├── Login.jsx
│   │   ├── VideojuegoCard.jsx
│   │   ├── PlataformaSection.jsx
│   │   ├── Buscador.jsx
│   │   ├── FavoritoButton.jsx
│   │   └── ...
│   ├── pages/
│   │   ├── Home.jsx
│   │   ├── PlataformaPage.jsx
│   │   ├── VideojuegoDetail.jsx
│   │   └── NotFound.jsx
│   ├── context/
│   │   └── AppContext.jsx
│   ├── hooks/
│   │   ├── useAuth.js
│   │   └── useFavoritos.js
│   ├── utils/
│   │   └── ...
│   ├── App.jsx
│   └── main.jsx
└── package.json
```

Evaluación

1. **Funcionalidad**: La aplicación debe funcionar correctamente en React.
2. **Uso de Hooks**: Se evaluará el uso adecuado de `useState`, `useEffect`, `useParams`, `useNavigate`, `useContext`, y custom hooks.
3. **Modularidad**: El código debe estar bien organizado y estructurado en componentes reutilizables.
4. **Estado Global**: El estado de autenticación y favoritos debe manejarse correctamente con `useContext` y reflejarse en todas las páginas.
5. **Manejo de errores**: Implementación correcta de la página 404 y manejo de errores en las peticiones fetch.
6. **Carga de datos**: Correcta implementación del fetch a los archivos JSON desde la carpeta `public/data/`.
7. **Diseño**: Mantener la estética gaming (azul oscuro, verde neón y blanco) y la funcionalidad responsive.

Entrega

Los alumnos deberán subir el proyecto a un repositorio de **GitHub**, con instrucciones claras para ejecutarlo, y enviar el enlace antes de la fecha límite.

Rúbrica de Evaluación

Puntaje Total: 100 puntos

(Nota: Los puntos se redistribuyeron después de eliminar useMemo y useCallback como requerimientos obligatorios)

1. React Router - Navegación (10 puntos)

Criterio	Excelente (9-10)	Bueno (7-8)	Regular (4-6)	Insuficiente (0-3)
Implementación de rutas	Todas las rutas funcionan correctamente: `/` (Home), `/plataforma/:nombre` (Plataforma), `/videojuego/:id` (Detalles), `*` (404). Navegación fluida con `useNavigate` y `useParams` usado correctamente	Rutas principales funcionan, puede faltar ruta de plataforma o tener errores menores en navegación	Rutas implementadas pero con errores significativos en la navegación o falta `useParams`	No implementa react-router o rutas no funcionales

Puntos clave a evaluar:

- ✅ Instalación de `react-router-dom`
- ✅ Configuración de `BrowserRouter`
- ✅ Ruta principal `/` funcional
- ✅ Ruta dinámica `/plataforma/:nombre` funcional con `useParams`
- ✅ Ruta dinámica `/videojuego/:id` funcional con `useParams`
- ✅ Ruta 404 `*` implementada
- ✅ Uso correcto de `useNavigate()` para navegación programática
- ✅ Uso correcto de `useParams()` para leer parámetros de URL

---

2. useState - Manejo de Estado (10 puntos)

Criterio	Excelente (9-10)	Bueno (7-8)	Regular (4-6)	Insuficiente (0-3)
Uso de useState	`useState` usado correctamente en múltiples componentes: lista de videojuegos, detalles, estados de carga, búsqueda, favoritos. Procesamiento correcto de estructura agrupada	`useState` implementado correctamente con algunos errores menores	`useState` usado pero con problemas en el procesamiento de datos	No usa `useState` o su uso es completamente erróneo

Puntos clave a evaluar:

- ✅ Estado para lista de videojuegos en Home (procesar estructura agrupada)
- ✅ Estado para término de búsqueda
- ✅ Estado para detalles de videojuego en VideojuegoDetail
- ✅ Estados de carga (loading) implementados
- ✅ Procesamiento correcto de estructura agrupada por plataforma
- ✅ Manejo correcto de la actualización del estado

---

3. useEffect y useParams - Efectos y Parámetros de URL (12 puntos)

Criterio	Excelente (11-12)	Bueno (9-10)	Regular (5-8)	Insuficiente (0-4)
Implementación de useEffect y useParams	`useEffect` usado correctamente para cargar datos. `useParams` usado correctamente en PlataformaPage y VideojuegoDetail para leer parámetros de URL. Procesamiento correcto de estructura agrupada	`useEffect` y `useParams` implementados con errores menores	`useEffect` o `useParams` usado pero con problemas significativos	No usa `useEffect` o `useParams` o su uso genera errores críticos

Puntos clave a evaluar:

- ✅ Carga de videojuegos desde `/data/videojuegos.json` (estructura agrupada por plataforma)
- ✅ Procesamiento correcto de la estructura del objeto agrupado por plataforma
- ✅ Carga de videojuego individual en VideojuegoDetail usando `useParams` (buscar en todas las plataformas)
- ✅ Extracción de videojuegos por plataforma en PlataformaPage usando `useParams` (acceder a `data[plataforma]`)
- ✅ Array de dependencias correcto en `useEffect`
- ✅ Manejo de errores en peticiones fetch
- ✅ Estados de carga mientras se obtienen datos

---

4. useContext - Estado Global (Autenticación y Favoritos) (13 puntos)

Criterio	Excelente (11-13)	Bueno (9-10)	Regular (5-8)	Insuficiente (0-4)
Context API implementado	`AppContext` creado y usado correctamente. Maneja autenticación Y favoritos. Estado disponible en toda la app. Login, logout, agregar/quitar favoritos funcionan perfectamente	Context implementado, funciona pero con errores menores en favoritos o autenticación	Context creado pero con problemas significativos en su implementación o uso	No implementa `useContext` o no funciona

Puntos clave a evaluar:

- ✅ Archivo `AppContext.jsx` creado (o `AuthContext.jsx` y `FavoritosContext.jsx` combinados)
- ✅ Provider envolviendo la aplicación
- ✅ Estado de usuario Y favoritos compartidos globalmente
- ✅ Funciones `login`, `logout`, `agregarFavorito`, `quitarFavorito`, `esFavorito` disponibles
- ✅ Componente Login actualizado según estado
- ✅ Estado persiste durante la navegación
- ✅ Favoritos se muestran visualmente en cards y detalles

---

5. Custom Hooks - useAuth y useFavoritos (10 puntos)

Criterio	Excelente (9-10)	Bueno (7-8)	Regular (4-6)	Insuficiente (0-3)
Custom Hooks implementados	Ambos hooks (`useAuth` y `useFavoritos`) implementados correctamente. Encapsulan lógica apropiada, usan el contexto correctamente, retornan funciones y estados necesarios	Hooks creados y funcionales con errores menores	Un hook creado pero el otro no, o ambos con errores significativos	No implementa custom hooks o no funcionan

Puntos clave a evaluar:

- ✅ Archivo `useAuth.js` en carpeta `hooks/`
- ✅ Archivo `useFavoritos.js` en carpeta `hooks/`
- ✅ Lógica de login encapsulada en `useAuth`
- ✅ Lógica de favoritos encapsulada en `useFavoritos`
- ✅ Validación contra `usuarios.json`
- ✅ Retornan funciones y estados necesarios
- ✅ Reutilizables y siguiendo convenciones de hooks

---

6. Página Principal - Home (9 puntos)

Criterio	Excelente (8-9)	Bueno (6-7)	Regular (4-5)	Insuficiente (0-3)
Componente Home funcional	Muestra todos los videojuegos correctamente organizados por plataformas, componente de búsqueda funcional, filtros por plataforma	Muestra videojuegos y búsqueda funciona con errores menores	Muestra videojuegos pero falta búsqueda o filtros	No muestra videojuegos o no funciona

	navegan correctamente, cards muestran favoritos, diseño atractivo			

Puntos clave a evaluar:

- ✅ Renderiza lista completa de videojuegos
- ✅ **Componente Buscador implementado y funcional** (obligatorio)
- ✅ Videojuegos organizados por plataformas
- ✅ Filtros por plataforma navegan a `/plataforma/:nombre`
- ✅ Cards muestran indicador de favoritos
- ✅ Click en videojuego navega a detalles
- ✅ Diseño responsive
- ✅ Manejo de estado de carga

---

7. Página de Plataforma - PlataformaPage (8 puntos) - NUEVO

Criterio	Excelente (8)	Bueno (6-7)	Regular (4-5)	Insuficiente (0-3)
Componente PlataformaPage funcional	Usa `useParams` correctamente, filtra videojuegos por plataforma, muestra solo los de esa plataforma, valida plataforma existente, botón volver funciona	Funciona pero con errores menores en validación o filtrado	Implementado pero con problemas significativos	No implementa esta página

Puntos clave a evaluar:

- ✅ Usa `useParams()` para obtener nombre de plataforma
- ✅ Filtra videojuegos por plataforma correctamente
- ✅ Valida que la plataforma existe
- ✅ Muestra mensaje si plataforma no existe
- ✅ Botón "Volver al inicio" funcional
- ✅ Manejo de estado de carga

---

8. Página de Detalles - VideojuegoDetail (9 puntos)

Criterio	Excelente (8-9)	Bueno (6-7)	Regular (4-5)	Insuficiente (0-3)
Componente VideojuegoDetail funcional	Usa `useParams` correctamente, muestra todos los detalles, botón de favorito funcional usando contexto, botón volver funciona	Muestra información principal pero falta botón favorito	Muestra información pero con problemas significativos	No muestra detalles o no funciona

Puntos clave a evaluar:

- ✅ Uso de `useParams()` para obtener ID
- ✅ Carga de datos de videojuego específico
- ✅ Muestra imagen, título, desarrollador, descripción
- ✅ Muestra año, género, calificación
- ✅ **Sección destacada de DLCs incluidos**
- ✅ Muestra requisitos del sistema
- ✅ Muestra modos de juego y características
- ✅ **Botón de favorito funcional** (obligatorio)
- ✅ Botón "Volver al catálogo" funcional
- ✅ Manejo de videojuego no encontrado

---

9. Página 404 - NotFound (5 puntos)

Criterio	Excelente (5)	Bueno (4)	Regular (2-3)	Insuficiente (0-1)

Componente NotFound	Página 404 completa con logo GAMEHUB (🎮), mensaje de error 404, descripción amigable temática, botón de retorno funcional, estética consistente	Página 404 funcional con elementos menores faltantes	Página 404 básica sin estética o funcionalidad completa	No implementa página 404

Puntos clave a evaluar:

- ✅ Logo de GAMEHUB visible (🎮)
- ✅ Mensaje "404 - Página no encontrada"
- ✅ Descripción amigable del error temática
- ✅ Botón "Volver al inicio" con `useNavigate()`
- ✅ Estética consistente (colores gaming)

---

10. Sistema de Autenticación (8 puntos)

Criterio	Excelente (8)	Bueno (6-7)	Regular (4-5)	Insuficiente (0-3)
Login funcional	Sistema de login completo: formulario funcional, valida contra `usuarios.json`, muestra errores, actualiza UI mostrando nombre completo y rol, botón de logout funciona	Login funciona con errores menores en UI	Login implementado pero con problemas de validación o UI	Login no funciona o no implementado

Puntos clave a evaluar:

- ✅ Formulario de login (usuario y contraseña)
- ✅ Validación contra `usuarios.json`
- ✅ Mensajes de error informativos
- ✅ UI actualizada al iniciar sesión
- ✅ Botón de "Cerrar sesión" funcional
- ✅ Estado persistente durante navegación

---

11. Sistema de Favoritos (6 puntos) - NUEVO

Criterio	Excelente (6)	Bueno (4-5)	Regular (2-3)	Insuficiente (0-1)
Favoritos funcional	Sistema de favoritos completo: agregar/quitar funciona, se muestra visualmente en cards y detalles, persiste durante navegación, usa contexto correctamente	Favoritos funcionan con errores menores en visualización	Favoritos implementados pero con problemas significativos	No implementa favoritos

Puntos clave a evaluar:

- ✅ Agregar videojuego a favoritos funciona
- ✅ Quitar videojuego de favoritos funciona
- ✅ Indicador visual en cards de favoritos
- ✅ Botón de favorito en página de detalles
- ✅ Favoritos persisten durante navegación
- ✅ Usa contexto correctamente

Criterios de Desaprobación Automática

El proyecto será desaprobado automáticamente si:

- ❌ La aplicación no ejecuta o tiene errores críticos que impiden su funcionamiento
- ❌ No se implementa el sistema de plataformas de videojuegos
- ❌ No se implementa la página de plataforma (`/plataforma/:nombre`)
- ❌ No se implementa el componente de búsqueda
- ❌ No se usa `useParams` en las rutas dinámicas

Bonus (Hasta 10 puntos adicionales)

Puntos extras por implementaciones adicionales:

- **+3 puntos**: Persistencia de favoritos con localStorage
- **+4 puntos**: Filtros adicionales por género con navegación a `/genero/:nombre`
- **+3 puntos**: Página de favoritos (`/favoritos`) que muestre todos los videojuegos favoritos del usuario

---

Notas Adicionales

Temática Gaming

La aplicación debe mantener la identidad visual inspirada en el mundo gaming:

- **Colores principales**: Azul oscuro (#0A0E27), Verde neón (#00FF88), Blanco (#FFFFFF)
- **Logo**: 🎮 GAMEHUB
- **Slogan**: "Tu catálogo de videojuegos definitivo"

Datos Importantes

- Cada videojuego incluye **DLCs específicos** con información detallada
- Los videojuegos están organizados en **4 plataformas**: PC, PlayStation, Xbox y Nintendo
- Cada videojuego tiene información de: desarrollador, año, género, calificación, requisitos del sistema, modos de juego y características
- Los usuarios tienen: nombre, apellido, username, password, rol y email

Diferencias Clave con el Examen Original

1. **Estructura JSON diferente**: El JSON viene agrupado por plataforma como objeto, no como array plano.
2. **Rutas adicionales**: Se requiere una página de plataforma (`/plataforma/:nombre`) que no existía en el examen original.
3. **useParams obligatorio**: Debe usarse en múltiples componentes, no solo sugerido.
4. **Sistema de favoritos**: Es obligatorio y debe integrarse con el contexto.
5. **Componente de búsqueda**: Es obligatorio, no opcional.
6. **Custom Hook adicional**: Se requiere `useFavoritos` además de `useAuth` .
7. **Contexto combinado**: El contexto debe manejar autenticación Y favoritos juntos.

Recomendaciones

1. Comienza configurando el routing básico con todas las rutas necesarias
2. Implementa el contexto temprano para tener autenticación y favoritos disponibles
3. Procesa correctamente la estructura JSON agrupada por plataforma desde el principio
4. Crea el componente de búsqueda como un componente separado y reutilizable
5. No olvides validar que las plataformas existen en la página de plataforma
6. Asegúrate de que los favoritos se muestren visualmente en todas las páginas relevantes
7. Para buscar un videojuego por ID, recuerda usar `Object.values(data).flat().find(...)` para buscar en todas las plataformas